

Django

Why do we use Django? Django is a high level Python Framework used for rapid development of secure and maintainable websites

Django MTV structure allows us to outsource the different kinds of tasks to specific file locations.

Django Files Roles:

- Routes (expected requests).
- Functions associated with those routes (how our server responds)
- Database interaction (Storing & retrieving data from database)
- Templates (what the user interacts with)

* Django further divides an application into individual apps that work together as a whole project

(MTV): Model Template View

MTV is the Django's pattern for organizing the code, similar to MVC.
Model View Controller

Responsibilities:

Django Component	MVC Component	Purpose
Model	Model	• May build database tables • Handles logic that relies on data • Interfaces with the database
Template	View	• HTML page that gets served to the client • It may contain some logic to be handled by a template engine.

CREATE a Project:

- `django-admin startproject name`
- `python manage.py runserver`
- `python manage.py startapp name`
- `settings.py` ⇒ add app
- `proj/proj/urls` ⇒
`path("", include("app-name.urls"))`
- `proj/app/urls` ⇒
`path("", views.index),`
- `views.py` ⇒ add methods

Django Component	MVC Component	Purpose
View	Controller	• Receives incoming requests. • Minimal logic • Calls on models to aggregate/process data • Determines appropriate response.

Django Virtual Environment:

Create your environment in the my-environments folder

Mac: `python3 -m venv djangoPy3Env`

activate it → `source djangoEnvPy3/bin/activate`

install django → `pip install Django == 2.2.4`

Modularization: Dividing code into smaller independent files or sections so that each handles specific task.

NOTE

In Django: It applies modularization through its MTV structure (Model-Template-View)

Model: handles database & data logic.

Template: handles what the user sees.

View: handles user requests and responses

Each of those is a module, stored in separate files

Remember:

A single application in Django (in this case, every assignment), is called a project containing one or more apps.

How to create a Django Project:

[1] With our Django virtual environment activated, create a new Django project.

→ navigate to where you want the project to be saved

→ run this command:

```
django-admin startproject project-name
```

[2] A new Django project has been created, let's run it:

```
python3 manage.py runserver
```

[3] Open localhost:8000 on your browser.

[4] For every app we want to create we'll run this:

```
python3 manage.py startapp yourapp-name
```

NOTE The apps in project cannot have the same name as the project

[5] in the project file, find the settings.py file, find the variable INSTALLED_APPS and add your app name in it:

settings.py inside project-name/project-name/settings.py

```
INSTALLED_APPS = [
```

```
    'your-name-app-here'
```

```
] - - -
```

[6] Now, we'll be creating our Routes.

your-project / your-project / urls.py

```
from django.urls import path, include
```

```
urlpatterns = [
```

```
    path('', include('your-app-name.urls')),
```

```
]
```

[7] create a new urls.py file in your-app-name folder

your-project-name / your-app-name / urls.py

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [
```

```
    path('', views.index),
```

```
]
```

```
✓ MY-Project
  ✓ my-project
    →
    → urls.py
    → settings.py
  ✓ some-app
    ✓ migrations
    → apps.py
    → urls.py
    → views.py
  ✓ manage.py
```

[8] And then put a function called index in our app's views.py file:

your-project-name / your-app-name / views.py

```
from django.shortcuts import render, HttpResponse
```

```
def index(request):
```

```
    return HttpResponse(~this is equivalent of @app.route('/')")
```

[9] Now run project $\Rightarrow$ python manage.py runserver
your-project-name>

NOTE Do not manually change the name of any of your folders after creation.

Django Routing:

→ We have a new Django project running, but what code did we add there at the end.

project-name/project-name/urls.py

from django.urls import path, include

urlpatterns = [

path("", include('app-name.urls')),

]

→ The urlpatterns: is simply a variable containing list of URIs that this project recognises

2 arguments are being passed to the url functions:

1. A raw string representing a route pattern (in our example: "")
2. What to do if the pattern matches ("include('app-name.urls')").

→ These 2 arguments for the urls.py in the project folder

project-name/app-name/urls.py

from django.urls import path

from . import views

urlpatterns = [

path("", views.py)

→ Arguments indicate:

1. "" - the rest of the route both starts and ends with nothing
("/" is the entire route)

2. views.index - if the requested route matches this pattern
the function with the ~~same~~ name "index" from the app's views.py
file will be invoked

So, if the routes wants views.index, we better have one:

Project-name/app-name/views.py

from django.shortcuts import render, HttpResponse

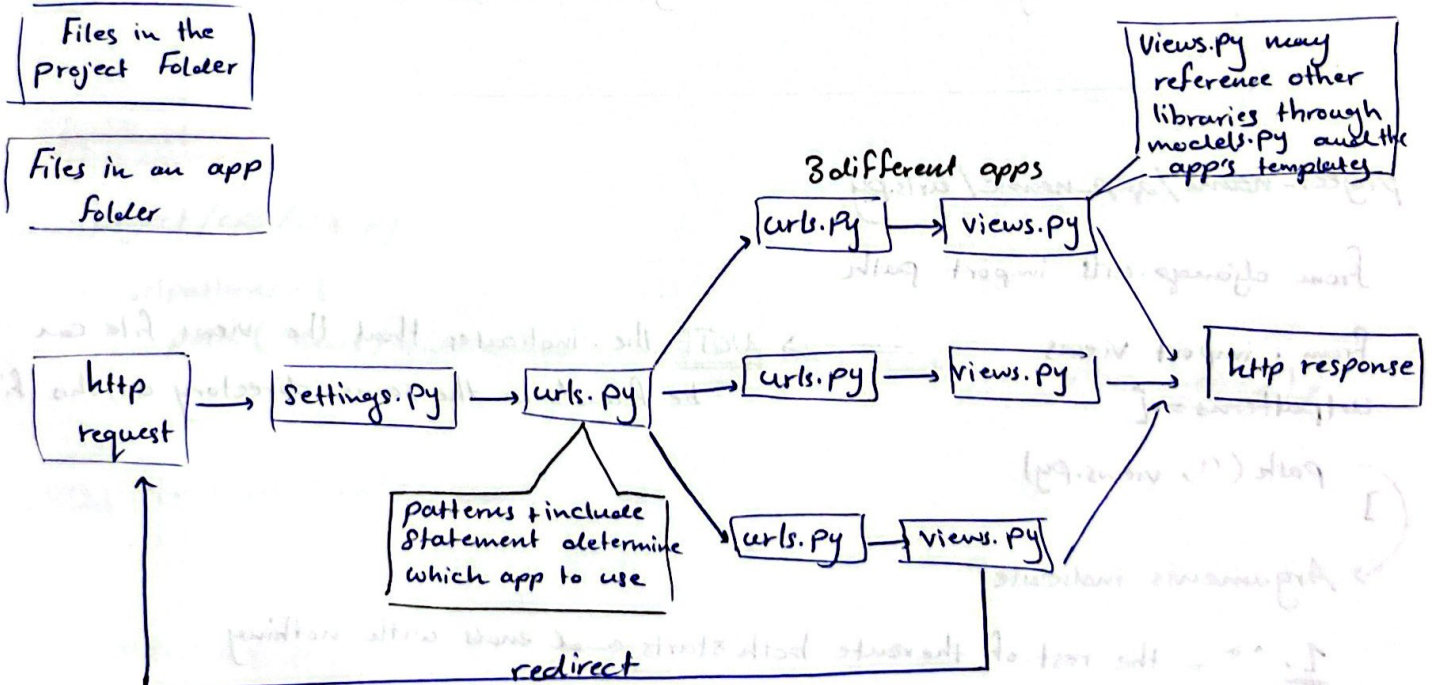
def index(request):

return HttpResponse (~ Response from index method from root route, localhost:8000)

NOTES!

- Every function's first argument will be the request object.
- We don't distinguish anywhere between GET vs POST requests in our routes. This will be done without a given function.
- If we're returning a string, we cannot simply return it, but we must send it via HttpResponse (which must be imported, we'll be returning rendered templates soon)

Django Project



Routing with Parameters:

Some-project/some-app/urls.py

```
urlpatterns = [  
    path("bears", views.one-method),  $\Rightarrow$  # would only match localhost:8000/bears  
    path("bears/<int: my-val>", views.another-method),  $\Rightarrow$  localhost:8000/bears/23  
    path("bears/<str: name>/poke", views.get-another),  $\Rightarrow$  local--/bears/pooh/poke  
    path("<int: id>/<str: color>", views.one-more)  $\Rightarrow$  localhost:8000/17/brown.
```

The corresponding functions:

```
def one-method(request):  $\Rightarrow$  no value passed via url  
    "  
def another-method(request, my-val):  $\Rightarrow$  my-val would be a val from the url  
    "  
def get_another(request, name):  $\Rightarrow$  name from the url  
    "  
def one-more(request, id, color):  $\Rightarrow$  id would be a number, and color  
    " a string from the URL
```

In Short

$\rightarrow$ project/app/urls.py

```
urlpatterns = [  
    path("bears/<int: my-val>/name<str: my-name>", views.method),  
]
```

url, localhost:8000/23/Now

$\rightarrow$ views.py

```
def method(request, my-val, my-name):
```

Response Types:

- HttpResponse: can be used to pass a string as a response.

- Redirect: Used to navigate to a different view method, before sending a final response to the client.

NOTE Even though we don't include the first / in our project urls.py file, when redirecting, you should provide the whole path starting with the first /.

- JSONResponse: used to return a JSON object.

Code

urls.py

```
urlpatterns = [  
    path('', views.root-method),  
    path('another-route', views.another-method),  
    path('redirected-route', views.redirected-method)  
]
```

views.py

```
from django.shortcuts import HttpResponse, redirect,
```

```
from django.http import JsonResponse
```

```
⇒ def root-method(request):
```

```
    return HttpResponse("String response from root-method")
```

```
⇒ def another-method(request):
```

```
    return redirect("/redirected-route")
```

```
⇒ def redirected-method(request):
```

```
    return JsonResponse({"response": "JSON response from redirected-method",
```

```
                        "status": True})
```

[8]

Response Types:

- HttpResponse: can be used to pass a string as a response.
- Redirect: Used to navigate to a different view method, before sending a final response to the client.

NOTE Even though we don't include the first / in our project urls.py file, when redirecting, you should provide the whole path starting with the first /.

- JSONResponse: used to return a JSON object.

Code

urls.py

```
urlpatterns = [  
    path('', views.root-method),  
    path('another-route', views.another-method),  
    path('redirected-route', views.redirected-method)  
]
```

views.py

```
from django.shortcuts import HttpResponse, redirect,  
from django.http import JsonResponse
```

```
⇒ def root-method(request):
```

```
    return HttpResponse("String response from root-method")
```

```
⇒ def another-method(request):
```

```
    return redirect("/redirected-route")
```

```
⇒ def redirected-method(request):
```

```
    return JsonResponse({"response": "JSON response from redirected-method",  
                        "status": True})
```

Templates:

Each app should have its own templates folder.

→ project/app/views.py

```
def index(request):  
    return render(request, "index.html")
```

NOTE When you call the render function, the first argument is always a request and the second argument is a string indicating which HTML file to render.

How can we pass data to the template?

by passing a single dictionary, whose keys will be the variable names available on the template

→ project/app/views.py

```
def index(request):  
    context = {  
        "name": "Noelle",  
        "favorite_color": "turquoise",  
        "pets": ["Bruce", "Fitz", "Georgie"]  
    }  
    return render(request, "index.html", context)
```

project/app/templates/index.html

```
<p> {{ name }} </p>  
<p> Pets: </p>  
<ul>  
    {% for pet in pets %}  
        <li> {{ pet }} </li>  
    {% endfor %}  
</ul>
```

NOTE You cannot use square brackets with Django's template engine! , Instead, use dot notation
→ array [0] becomes {{array.0}}

Static Files:

For each app we also need a static folder at the same level as our templates folder.

In templates, we have to add a line indicating we want to use our static files:

<head>

<meta charset="utf-8">

<title> Index </title>

{% load static %}

→ <link rel="stylesheet" href="{% static 'css/style.css' %}">

~~→ <link rel="stylesheet" href="{% static 'css/style.css' %}">~~

→ <script src="{% static 'js/script.js' %}"></script>

→

GET Vs. POST requests:

• A Django view method will be called regardless of the type of HTTP Request method. We can check the request method type being received like so:

⇒ def some_function(request):

if request.method == "GET":

print("A get request is being made")

return render(request, "index.html")

if request.method == "POST":

print("A Post request is being made")

return ~~render~~ redirect("/")

Submitting Form Data:

- In Django we'll utilize the `request.POST` to access any form of data

from Django. Shortcuts import render, redirect

def some_function(request):

```
if request.method == "GET":  
    print (request.get)
```

```
if request.method == "POST":  
    print (request.POST)
```

CSRF-Token:

Any forms being submitted as POST requests must include a CSRF token:

CSRF-token $\Rightarrow$ $\ominus$ prevents cross-site request forgery

A malicious attack where a hacker can pretend to be another user and submit data to a site that recognizes that user.

$\ominus$ allows Django to add a hidden input field and value that helps our server recognize genuine requests

$\ominus$ if you forget to add it, Django will kindly provide a clear error message if you attempt to submit a form without a token

Add a token to each form with this line:

```
<form action="/some-route" method="post">  
    {% csrf_token %}  
    <input -- name="input-one">  
</form>
```

Remember:

That the names of the input fields from our form will be the keys we use to access the data on our server.

→ This is how we should retrieve data from form:

```
def some-function(request):
```

```
    if request.method == "POST":
```

```
        value-from-field-one = request.POST["one"]
```

POST Form Submission:

Code

→ form_test/form_app/views.py

from ---

def index(request):

return index(request, "index.html")

→ form_test/form_app/templates/index.html

<form action='/users' method='post'>

{% csrf_token %}

Name: <input name='name' type='text'>

</form>

POST Form Parts:

1 action attribute:

This route will process the form (not the one that shows the form)

2 method attribute:

Default is Get, but our options are POST & GET.

3 CSRF Token:

• This token is used to prevent cross-site request forgery, a malicious attack where a hacker pretends to be another user, and submits data to a site that recognizes that user.

• Adding it allows Django to add a hidden input field and value that helps our server recognize genuine requests.

• Forgetting it will through an error.

- What should happen when the form is submitted?

The POST request that is handled using the route /users

we'll add it to our app `urls.py`:

```
form-test / form-app / urls.py
urlpatterns = [
    path("", views.index),
    path('users', views.create_user)
]
```

⇒ Now let's set up our method

`form-test / form-app / views.py`

```
def index(request):
    return render(request, "index.html")
```

```
def create_user(request):
    print(request.POST)
    return render(request, "index.html")
```

NOTICE that the data is being sent to the ~~user~~ server in a dictionary

```
def create_user(request):
    name-from-form = request.POST['name']
    email-from-form = request.POST['email']
    return render(request, "index.html")
```

• Accessing Data: `request.POST['name-of-input']`

NOTE

- The type of anything coming in through request.POST will be a string, no matter what.
- We cannot render immediately as a response to a POST request.

Forms final code

1 index.html /

```
<form action="/users" method="post">
    {% csrf_token %}
    <input name="username" type="-->
    <input name="email" -->
</form>
```

2 app/views.py /

```
def index(request):
    return render(request, 'index.html')
```

def create-user(request):

```
name-on-form = request.POST['name']
email-on-form = request.POST['email']
```

```
⇒ context = [
```

```
    "name" = name-on-form,
```

```
    "email" = email-on-form
```

```
]
```

```
return render(request, "show.html", context)
```

3 show.html

```
{{ name name }}
```

```
{{ email email }}
```

3 project/app/urls.py

```
path(' ', views.index),
path('users', views.create-user)
```

MORE ON REDIRECTING:

Now, we have a problem: form data is used immediately in the next page but we have to add a step between showing and displaying the form data why? because after we got the data on the next page, if I hit refresh it will give me a pop up of Confirm post submission if I hit continue it will restore the same data in database again. or process the data again for example.

3 Steps: to solve this.

1. Set up a route and a method that will render a template to show the form.
2. Set up a route and a method ~~that~~ to process the form data that will redirect.
3. Set up a route and a method that will render a template indicating the form was successfully processed.

Code:

proj / app / urls.py

```
urlpatterns = [  
    path("", views.index),  
    path('create-user', views.create-user),  
    path('success', views.success)  
]
```

proj / app / views.py

```
def index(request):  
    return render(request, "index.html")
```

```
def create-user(request):  
    name-from-form = request.POST['name']  
    email-from-form = request.POST['email']
```

```
Context = [  
    'name' = name-on-form,  
    'email' = email-on-form  
]  
return redirect("/success")
```

```
def success(request):  
    return render("show.html")
```

Session:

• HTTP response/request cycle is stateless.

• State: Current data/condition of a program at a specific moment.

↳ Remembering across requests

• Stateless: - The server doesn't remember anything about the client between requests
- Each HTTP method is independent and doesn't carry any memory for previous ones.

• How can sites remember your data in shopping cart?

Using persistent data storage $\Rightarrow$ Session

Web pages
independent
&
ignorant

Session: lets a web browser remember information about a user between multiple HTTP requests.

Why sessions are useful?

- It allows a web browser to store helpful data like:

This
memory is
State

- Whether a user is logged in.
- Who the current user is.
- What pages or links the user has visited.

NOTES

- A session stores data on the server to create a smoother, more personalized user experience.
- It bridges the gap between stateless HTTP protocols and stateful user experience.
- Developers decide what data to save in the session to use it later in future requests.

In Short Sessions make a stateless HTTP remember user information across different requests

How Django does it?

In Django, this memory / state is handled by the server, using sessions.

How it works?

- When you login, Django creates a session ID (a random key).
- That ID is stored in your browser as a cookie.
- Django keeps the actual session data (like username, userID) in one of those places:
 - the database (default: django_session table)
 - the cache (Redis, Memcached)
 - files on the server

So data is stored on the ~~browser~~ server not on the browser
only the session ID is stored in the browser.

Example:

request.session['username'] = 'NOUR'

Django saves this in the session table, and the user gets a small cookie like:

session id = 3fg89achd9ag

Cookies: small pieces of data that a server (like Django) sends to the browsers and the browser saves them locally.

- Packet of information from server to client.

Migrations: help manage out database's state, including creating & updating tables.

Using sessions in Django:

1. Django uses the Database to manage sessions.

- Django doesn't store ~~data~~ your session data like (user-id or cart-items) directly in the browser.

- Instead, it stores all the data in the database, inside a special table called `django-session`.

- To create that table, you need to run the migration command:

```
python manage.py migrate
```

- When you do that, Django applies a migration like:

⇒ Applying sessions.0001-initial.. OK

- This means that Django created a table to store session data.

2. Using sessions in `views.py`:

Once sessions are ~~applied~~ enabled, you can use them like a dictionary (Key-value pairs)

ex `def some-function (request):`

```
    request.session ['name'] = request.POST ['name']
```

```
    request.session ['counter'] = 100
```

⇒ This data is stored on the server (in the database) not in the browser

The browser only keeps a cookie with the session ID to find this data later.

3. Accessing Session Data in Templates

In your HTML templates, you can display session data like this:

```
<P> Name in session is: {{ request.session.name }} </P>
```

flask → 
السيرفر يتلقى الـ session ID المشفرة
encryption

4. So What's happening behind the scenes:

1. You run `python manage.py migrate` → Django creates the session table.
2. You save the data with `request.session['Key'] = value`.
3. Django saves it in the database, and sends a small cookie (session ID) to the browser.
4. When the user comes back, the browser sends that cookie. Django finds the correct session in the database.
5. You can access it with `request.session[...]`.

In short:

- The migrate command makes Django ready to store session data in the database.
- The `request.session` dictionary lets you save and retrieve data for each user session.

Session methods:

• `request.session['Key']`

→ This will retrieve (get) the value associated with "Key".

• `request.session['Key'] = 'value'`

→ Set the value that will be stored by "Key" to "value"

• `"Key" in request.session`

→ Returns a boolean of whether a key is in session or not.

• `{ session.session.name }`

→ Use dot notation (.) to access `request.session` keys from templates. Since square brackets aren't allowed there.

• `del request.session['Key']`

→ Deletes a session key if it exists, throws a KeyError if it doesn't, use along with `try` and `except` since it's better to ask for forgiveness than permission.

NOTE: If you're storing a list in a session that is being modified (such as an append) you will need to save the session after the append

→ `request.session['my-list'] = []`

→ `request.session['my-list'].append(new-item)`

→ `request.session.save()`

Hidden Inputs:

Hidden inputs: - form fields that are hidden from the user.

- Used to transfer information between different pages.
- an ordinary element without visual representation in the rendered HTML.

Syntax: `<input type="hidden" name="action" value="register">`

Why do we use them?

because we can have multiple forms in the same page with the same POST and /process route. So we can know which form was submitted, each form have a hidden input with the same name but different values

```
{
  if request.POST["which-form"] == 'register':
    // do registration process
  elif request.POST['which-form'] == 'login':
    // do login process
}
```

Named Routes:

Named Routes: make referencing our Django routes pretty easy.

→ Pass a keyword variable (name) to the url

ex from django.urls import path
from . import views

```
urlpatterns = [
    path("", views.toindex, name='my-index'),
    path('this-app/new', views.new, name='my-new'),
]
```

how to use them: instead of:

```
<a href="/target/this-app/new/"></a>
```

you can write:

```
<a href="{% url 'my-new' %}"></a>
```

Benefits:

- Easier to update URLs later (you only change them in one place)
- No risk of typos when reusing URLs across multiple templates.

Passing Arguments to Named Arguments:

If routes include a variable like `<int:id>` you can pass it like this

⇒ `<form action = "{% url 'my-delete' id=5 %}" method = 'post' >`
`<input type = 'submit' value = 'Submit' >`

HTTP Request

How Django Handles a Request?

1. settings.py:

- holds the project settings (security / middleware)
- Defines which middleware and routing files are active.

2. urls.py (project)

- Master routing file
- Delegates each request to the correct app's urls.py

3. urls.py (app)

- Defines URL patterns specific to that app and connects them to views.

4. views.py

- Handles the logic of what to do with the request
- Can import (models / templates / libraries)

must return one of the following:

- **HTTPResponse:** sends a plain response (text / HTML)
- **render:** renders and returns an HTML template
- **redirect:** redirects the user to another URL.

Flow Summary

Client → URL → settings.py → project urls.py → app urls.py → views
→ Response

* When starting a new project:

running `django-admin startproject` creates:

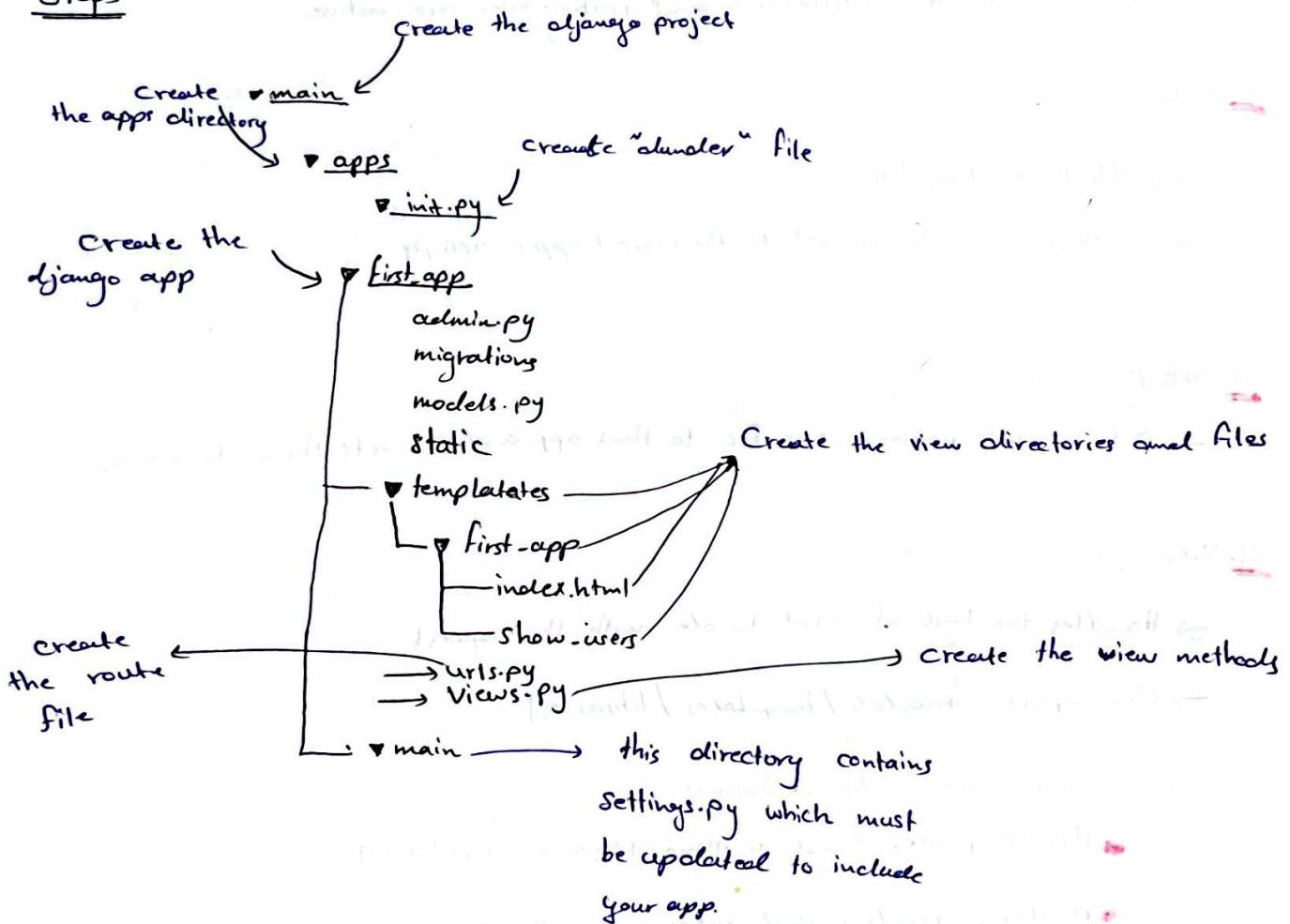
`Settings.py`

`main urls.py`

other files (apps, models, templates)

must be created manually.

Steps



Django URLs

- Django urls define how different URLs are connected to specific views in app. Functions

Setting Up URL Routing:

- Project-level urls.py (main/urls.py)

- The main routing app

- include() → helps connect the project URLs with specific app URLs

- App-level-urls.py

- from django.conf.urls import url : gives access to the url() function

Regular Expressions

Regex

Regex: rules for identifying or matching strings

Cheat Sheet

- ^ start of a string

^a → must begin with a

- \$ end of a string

a\$ → must end with a

- [] range or set

[0,9] ⇒ any digit between 0-9

- {n} n repetitions

[0,9]{2} ⇒ two digits between 0-9

- \d any digit

same as [0,9]

- \d+ ~~any word character~~
one or more digits

\d+ matches 9, 99, 1234 → += 1 or more times

- \w any word character

\w matches A, 3 ⇒ letters/numbers

- \w+ one or more word characters

\w+ matches hello, user123 ⇒ continuous letters/numbers

Forms

User type of requests:

Get	POST
• Typing in the URL • Click a link (HTML anchor tag) • Submitting a form with method = GET	• Submitting the form with the method POST